

Department of Computer Science and Engineering

B.Tech III Semester

Academic Year 2026-2027

Database Management Systems(2501IT05)

Capstone Project Documentation





SCHOOL OF COMPUTING

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Cornerstone project work entitled **“University Academic Management System”** is submitted by

- **T. Lakshmi Anvitha** (26B21CS079)
- **S. Keerthi** (26B21CS076)
- **P. Roshitha** (25B11CS749)
- **G. Deepthi** (25B11CS268)

in partial fulfillment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering for the academic year 2026-2027.

Project Guide

Mrs. K. Vydehi, M.Tech.,(Ph.D),
Assistant Professor,
Department of CSE

Head of the Department

Dr. Phani Sridhar Addepalli, Ph.D,
Assistant Professor,
Department of CSE

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Mrs. K Vydehi, Assistant Professor**, who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. Chandra Sekhar Kolli, Associate Professor, Dr Hari Jyothula, Associate Professor and Dr Pennada Siva Satya Prasad, Assistant Profeesor**, Department of Computer Science and Engineering, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. Phani Sridhar Addepalli, Assistant Professor**, for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

TABLE OF CONTENTS

Serial Number	Title	Page Number
1	Cover Page	1
2	Certificate	2
3	Acknowledgement	3
4	Table of Contents	4
5	Abstract	5
6	List of figures	6
7	List of Tables	7
8	Acronyms	8
9	Introduction	9
10	System Analysis	10
11	Database Design	12
12	ER Diagram	13
13	Relational Schema	14
14	Table Design	15
15	Normalization	17
16	SQL Implementation	20
17	SQL Queries & Screenshots	24
18	Conclusion & Future Enhancements	30
19	References	31

ABSTRACT

The **University Academic Management System** is a database management system designed to efficiently manage and organize the academic activities and information of a university. In a university, large amounts of data related to students, faculty, departments, courses, enrollments, and examinations need to be stored, maintained, and accessed regularly. Managing this information manually can lead to data redundancy, inconsistency, errors, and difficulty in retrieving required information. The proposed system provides a centralized and structured database to overcome these problems.

The system maintains important student details such as student ID, name, contact information, department, and academic records. It also manages faculty details, department information, course details, student enrollment, and examination records. The system establishes relationships between different entities to ensure that the data is properly connected and can be retrieved efficiently.

Primary keys, foreign keys, constraints, and relationships are used to maintain data integrity and consistency.

The system is developed using **Database Management System (DBMS) concepts** and includes operations such as creating tables, inserting records, updating data, deleting records, and retrieving information using SQL queries. It also applies **normalization up to Third Normal Form (3NF)** to reduce redundancy and avoid insertion, deletion, and update anomalies. The system can generate useful information such as enrolled courses, examination details, student academic records, and department-wise information.

Overall, the **University Academic Management System** provides a reliable, organized, and efficient approach for managing university academic data. It reduces manual effort, improves data accuracy, ensures data security and consistency, and makes information retrieval faster and easier. The system can also be extended in the future by adding features such as attendance management, results and grade calculation, timetable management, fee management, notifications, and student performance analysis.

LIST OF FIGURES

Figure 1. ER Diagram of University Academic Management Database

Figure 2. Database Table Relationship Diagram

Figure 3. DDL Operations-Table Creation.

Figure 4. DML Operations - Insertion.

Figure 5. DQL - Data Retrieval Using SQL Queries.

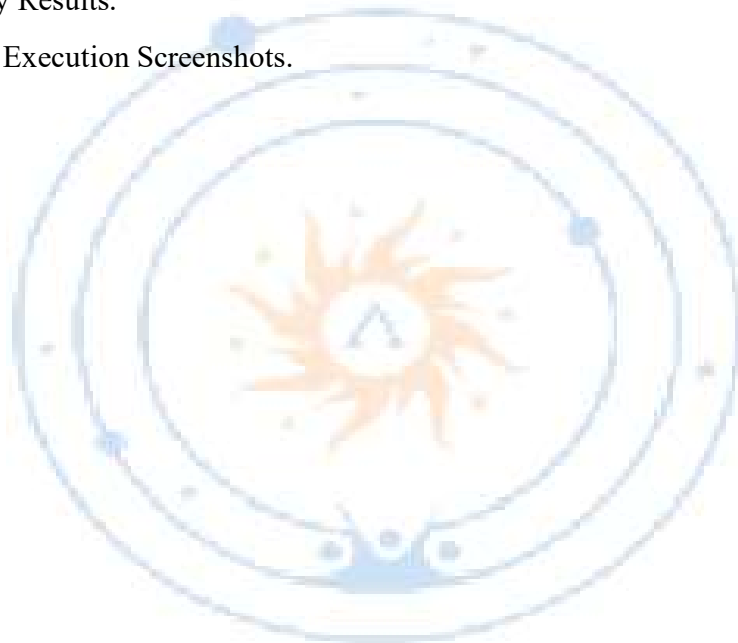
Figure 6. CRUD Operations.

Figure 7. SQL Joins and Aggregate Functions.

Figure 8. Views and Data Integrity Constraints.

Figure 9. SQL Query Results.

Figure 10. Database Execution Screenshots.



LIST OF TABLES

Table 1. Major Database Entities

Table 2. Department Table Structure

Table 3. Student Table Structure

Table 4. Faculty Table Structure

Table 5. Course Table Structure

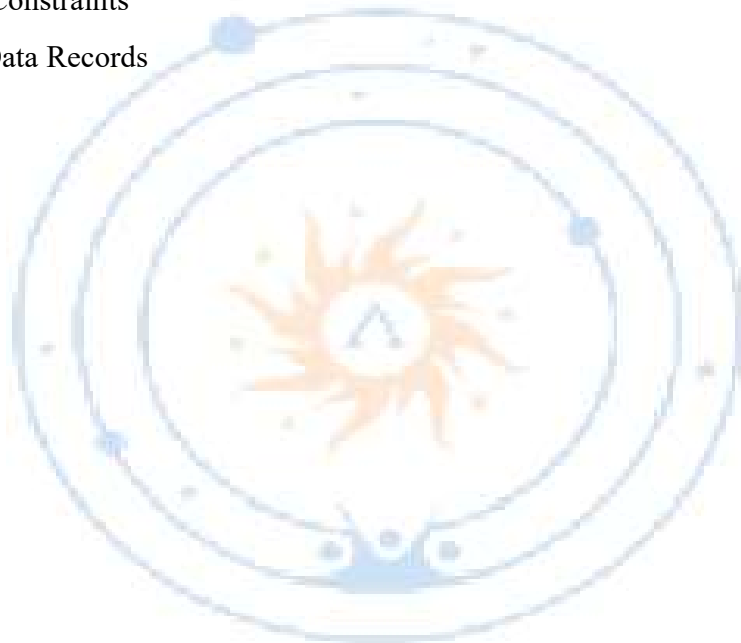
Table 6. Enrollment Table Structure

Table 7. Exams Table Structure

Table 8. Result Table Structure

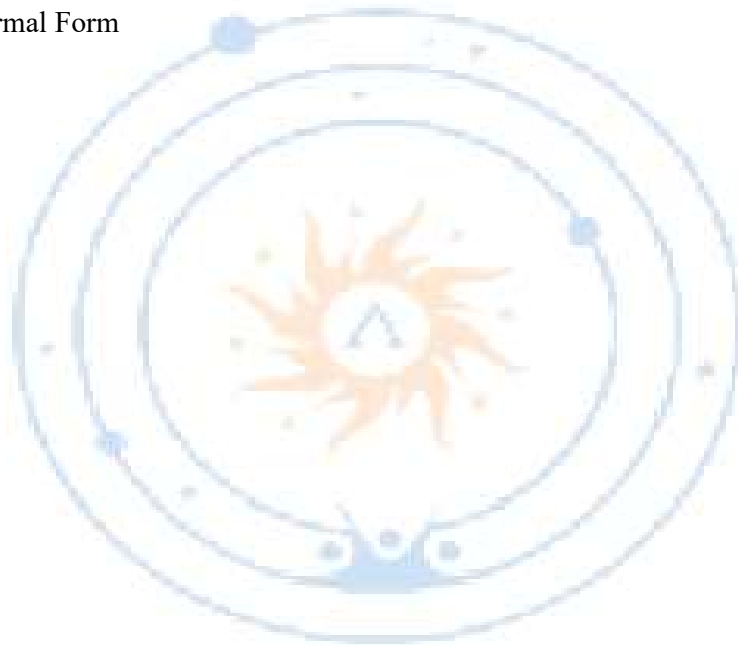
Table 9. Keys and Constraints

Table 10. Sample Data Records



ACRONYMS

- DBMS – Database Management System
- ER – Entity Relationship
- SQL – Structured Query Language
- DDL – Data Definition Language
- DML – Data Manipulation Language
- DQL – Data Query Language
- DCL – Data Control Language
- TCL – Transaction Control Language
- PK – Primary Key
- FK – Foreign Key
- 1NF – First Normal Form
- 2NF – Second Normal Form
- 3NF – Third Normal Form



1. INTRODUCTION

1.1 Brief Information about the Project

A University Academic Management System is a database application that manages academic information and administrative processes. The project provides a centralized relational database for departments, students, faculty, courses, enrollments, and examination results. It is intended to replace fragmented manual records and multiple spreadsheet files with a structured database that supports consistent storage and efficient retrieval.

1.2 Motivation and Contribution of Project

The project is motivated by common problems in academic record management such as duplicate records, time-consuming searches, inconsistent information, and difficulty in generating reports. A centralized database provides a systematic approach to storing related academic data and enforcing relationships through keys and constraints.

- Centralizing academic records in one relational database.
- Reducing duplicate and inconsistent information.
- Maintaining referential integrity using primary and foreign keys.
- Supporting faster academic queries and report generation.
- Providing a foundation for student, faculty, course, enrollment, and result management.

1.3 Objectives of the Project

- Develop a centralized academic database.
- Manage student records.
- Maintain faculty information.
- Store department details.
- Handle course registration and student enrollments.
- Record examination results.
- Reduce duplicate data and update anomalies.
- Maintain referential integrity.
- Support efficient SQL queries and academic reports.

1.4 Scope of the Project

The scope covers database-oriented management of university academic information. The core scope includes department, student, faculty, course, enrollment, and result data. The database can be used as the backend for an administrative application and can be extended later with attendance, fee, timetable, library, examination scheduling, and web/mobile interfaces.

2. SYSTEM ANALYSIS

2.1 Existing System

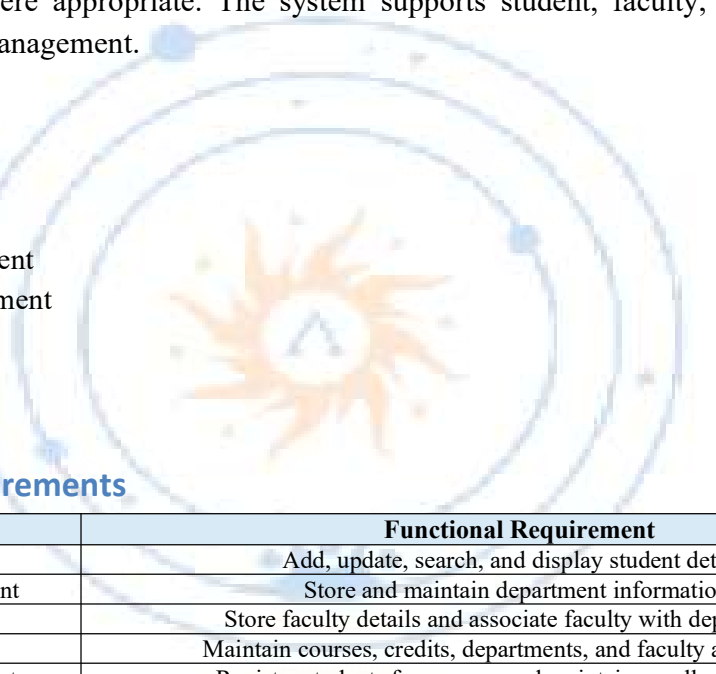
- Manual record maintenance.
- Multiple Excel sheets or disconnected files.
- Duplicate records and inconsistent information.
- Time-consuming searches.
- Difficulty in generating consolidated reports.
- Higher possibility of data-entry errors.

2.2 Proposed System

The proposed system uses a centralized relational database. It organizes academic information into related tables and uses primary keys, foreign keys, uniqueness rules, NOT NULL conditions, and CHECK constraints where appropriate. The system supports student, faculty, department, course, enrollment, and result management.

- Centralized database
- Student management
- Faculty management
- Course management
- Enrollment management
- Examination management
- Result processing
- SQL-based reporting
- Referential integrity

2.3 Functional Requirements



Module	Functional Requirement
Student Management	Add, update, search, and display student details.
Department Management	Store and maintain department information.
Faculty Management	Store faculty details and associate faculty with departments.
Course Management	Maintain courses, credits, departments, and faculty associations.
Enrollment Management	Register students for courses and maintain enrollment details.
Examination Management	Schedule examination, store exam details and maintain examination records.
Result Management	Store examination marks and grades associated with an enrollment.
Searching and reporting	Retrieve student, course, enrollment, and result information using SQL queries.

2.4 Non-Functional Requirements

- Performance – queries should return results efficiently for normal academic workloads.
- Integrity – invalid references and duplicate key values should be prevented.
- Security – database privileges should be controlled for administrative users.
- Usability – forms and reports should be understandable to users.
- Reliability – transactions should preserve consistent database state.
- Scalability – the design should support growth in students, courses, and academic records.

2.5 Requirements Specification

Category	Suggested Requirement
Hardware	Intel i3 or above; 4 GB RAM minimum; adequate storage; standard display and keyboard/mouse.
Operating System	Windows 10/11 or an equivalent supported platform.
Database	Oracle Database or another relational DBMS used by the implementation.



3. DATABASE DESIGN

3.1 Major Entities

Entity	Description
Department	Stores department information.
Student	Stores student records.
Faculty	Stores faculty details.
Course	Stores course information.
Enrollment	Represents student course registration.
Exams	Stores examination details such as exam date,type,and course.
Result	Stores examination result information linked to enrollment.

3.2 ER Model

The ER model represents the major entities and their relationships. The system includes entities such as Department , Student , Faculty , Course , Exam , Enrollment , and Result. The ER model defines relationships such as Department-Student , Department-Faculty , Department-Course , Faculty-Course , Student-Enrollment , Course-Enrollment , Course-Exam , and Enrollment-Result. The diagram also represents important attributes such as

Student_ID ,Roll_No ,Student_Name ,Department_ID ,Faculty_ID,Course_ID,and other academic details.

3.3 Relationships

Relationship	Description
Department → Student	One department can have many students.
Department → Faculty	One department can have many faculty members.
Department → Course	One department can offer many courses.
Faculty → Course	Faculty members can be associated with courses.
Student → Enrollment	A student can have multiple enrollment records.
Course → Enrollment	A course can have multiple enrollment records.
Course → Exam	A course can have multiple exams.
Enrollment → Result	An enrollment can have an associated result record.

4. ER DIAGRAM

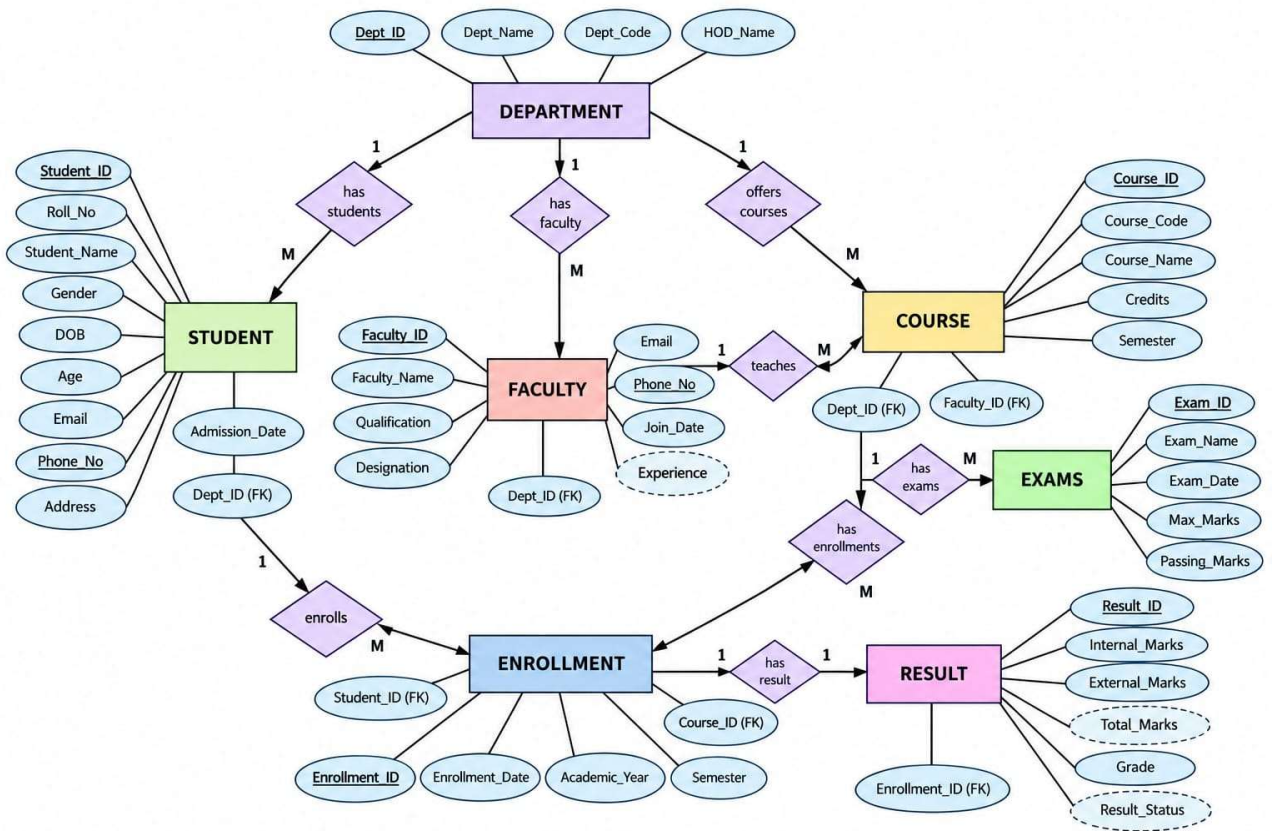


Figure 1: ER Diagram of University Academic Management Database

The ER diagram summarizes the entity structure and cardinalities described in the supplied DBMS project material. It is a documentation-level representation; the final project team should align attribute names and cardinalities with the actual implemented database.

5. RELATIONAL SCHEMA

Relation	Relational Schema
DEPARTMENT	Department_ID (PK), Department_Name, Department_Code, HOD_Name
STUDENT	Student_ID (PK), Student_Name, Roll_Number, Gender, DOB, Age, Email, Phone_No, Admission_Date, Address, Department_ID (FK)
FACULTY	Faculty_ID (PK), Faculty_Name, Qualification, Designation, Email, Phone_No, Join_Date, Experience, Department_ID (FK)
COURSE	Course_ID (PK), Course_Code, Course_Name, Credits, Semester, Department_ID (FK), Faculty_ID (FK)
Exams	Exam_ID (PK), Exam_Name, Exam_Date, Max_Marks, Passing_Marks, Course_ID (FK)
ENROLLMENT	Enrollment_ID (PK), Student_ID (FK), Course_ID (FK), Enrollment_Date, Academic_Year, Semester
RESULT	Result_ID (PK), Enrollment_ID (FK), Internal_Marks, External_Marks, Total_Marks, Grade, Result_Status

The relational schema follows the structure presented in the supplied project material. The Student and Faculty relations reference Department; Course references Department and Faculty; Enrollment references Student and Course; and Result references Enrollment.



6. TABLE DESIGN

6.1 Department Table

Attribute	Data Type	Constraint	Description
Dept_ID	INT / NUMBER	PK, NOT NULL	Unique department identifier
Dept_Name	VARCHAR	NOT NULL	Department name
Dept_Code	VARCHAR	UNIQUE	Department code
HOD_Name	VARCHAR	—	Head of Department name

6.2 Student Table

Attribute	Data Type	Constraint	Description
Student_ID	INT	PK	Unique student identifier
Student_Name	VARCHAR	NOT NULL	Student name
Roll_Number	VARCHAR	UNIQUE	Institution roll number
Gender	VARCHAR	—	Gender value as required by implementation
DOB	DATE	—	Date of birth
Age	INT	—	Age, if maintained
Email	VARCHAR	UNIQUE	Student email
Phone_No	VARCHAR	—	Contact number
Admission_Date	DATE	—	Admission date
Address	VARCHAR	—	Address
Department_ID	INT	FK	References Department

6.3 Faculty Table

Attribute	Data Type	Constraint	Description
Faculty_ID	INT	PK	Unique faculty identifier
Faculty_Name	VARCHAR	NOT NULL	Faculty name
Qualification	VARCHAR	—	Academic qualification
Designation	VARCHAR	—	Faculty designation
Email	VARCHAR	UNIQUE	Faculty email
Phone_No	VARCHAR	—	Contact number
Join_Date	DATE	—	Joining date
Experience	INT	—	Experience in years, if maintained
Dept_ID	INT	FK	References Department

6.4 Course Table

Attribute	Data Type	Constraint	Description
Course_ID	INT	PK	Unique course identifier
Course_Code	VARCHAR	UNIQUE	Course code
Course_Name	VARCHAR	NOT NULL	Course name
Credits	INT	CHECK	Credit value
Semester	INT	—	Semester offered
Dept_ID	INT	FK	References Department
Faculty_ID	INT	FK	References Faculty

6.5 Enrollment Table

Attribute	Data Type	Constraint	Description
-----------	-----------	------------	-------------

Enrollment_ID	INT	PK	Unique enrollment identifier
Student_ID	INT	FK	References Student
Course_ID	INT	FK	References Course
Enrollment_Date	DATE	—	Enrollment date
Academic_Year	VARCHAR	—	Academic year
Semester	INT	—	Semester

6.6 Exams Table

Attribute	Data Type	Constraint	Description
Exam_ID	INT	PK	Unique exam identifier
Exam_Name	VARCHAR	-	Name of the examination
Exam_Date	DATE	-	Date of the examination
Max_Marks	INT	-	Maximum marks for the exam
Passing_Marks	INT	-	Minimum marks required to pass
Course_ID	INT	FK	References Course

6.7 Result Table

Attribute	Data Type	Constraint	Description
Result_ID	INT	PK	Unique result identifier
Enrollment_ID	INT	FK	References Enrollment
Internal_Marks	DECIMAL	—	Internal assessment marks
External_Marks	DECIMAL	—	External examination marks
Total_Marks	DECIMAL	—	Total marks
Grade	VARCHAR	—	Grade obtained
Result_Status	VARCHAR	—	Pass/fail or equivalent status

6.7 Keys and Constraints

Constraint Type	Applied To / Purpose
Primary Key	Department_ID, Student_ID, Faculty_ID, Course_ID, Enrollment_ID, Result_ID
Foreign Key	Student.Department_ID; Faculty.Department_ID; Course.Department_ID; Course.Faculty_ID; Enrollment.Student_ID; Enrollment.Course_ID; Result.Enrollment_ID
NOT NULL	Required attributes such as names and identifiers
UNIQUE	Roll number, email, course code, and department code where applicable
CHECK	Values such as course credits and marks can be restricted to valid ranges

7. NORMALIZATION

Normalization organizes data into related relations to reduce redundancy and update anomalies. The supplied project template explicitly requires documentation from UNF to 3NF. The following example uses an academic registration record to illustrate the process.

7.1 Unnormalized Form (UNF)

A single academic record might contain Student_ID, Student_Name, Department_Name, Course_ID, Course_Name, Faculty_Name, multiple course values, and multiple result values in one record. Repeating course groups and repeated department/faculty information create redundancy.

7.2 First Normal Form (1NF)

In 1NF, repeating groups are removed and each attribute contains atomic values. Each course enrollment is represented as a separate row. Student and department details may still be repeated across multiple enrollment records.

Student_ID	Student_Name	Dept_ID	Dept_Name	Course_ID	Course_Name
101	Aarav Sharma	1	Computer Science	301	Programming in C
102	Ananya Reddy	2	Information Technology	302	DBMS
103	Rahul Kumar	3	Electronics	303	Digital Electronics

7.3 Second Normal Form (2NF)

In 2NF, partial dependencies on a composite key are removed. Student-specific attributes depend on Student_ID, course-specific attributes depend on Course_ID, and enrollment-specific attributes depend on Enrollment_ID. These dependencies motivate separate Student, Course, and Enrollment relations.

Table 1: STUDENT

Student_ID (PK)	Student_Name	Dept_ID	Dept_Name
101	Aarav Sharma	1	Computer Science
102	Ananya Reddy	2	Information Technology
103	Rahul Kumar	3	Electronics and Communication

Table 2: COURSE

Course_ID (PK)	Course_Name	Dept_ID
301	Programming in C	1
302	DBMS	2
303	Digital Electronics	3

Table 3: ENROLLMENT

Enrollment_ID (PK)	Student_ID (FK)	Course_ID (FK)
401	101	301
402	102	302
403	103	303

7.4 Third Normal Form (3NF)

In 3NF, transitive dependencies are removed. Department details are stored in Department rather than repeated in Student, Faculty, and Course. Faculty details are stored in Faculty rather than repeated in Course. Result details are linked to Enrollment through Result. The resulting structure supports the 3NF objective stated in the supplied project material.

Table 1: STUDENT

Student_ID (PK)	Student_Name	Dept_ID (FK)
101	Aarav Sharma	1
102	Ananya Reddy	2
103	Rahul Kumar	3

Table 2: DEPARTMENT

Dept_ID (PK)	Dept_Name	Dept_Code
1	Computer Science	CSE
2	Information Technology	IT
3	Electronics and Communication	ECE

Table 3: COURSE

Course_ID (PK)	Course_Name	Dept_ID (FK)
301	Programming in C	1
302	DBMS	2
303	Digital Electronics	3

Table 4: ENROLLMENT

Enrollment_ID (PK)	Student_ID (FK)	Course_ID (FK)
401	101	301
402	102	302
403	103	303

Relation	Reason for Separation
Department	Stores department-specific attributes once.
Student	Stores student-specific attributes and a Department_ID reference.
Faculty	Stores faculty-specific attributes and a Department_ID reference.
Course	Stores course-specific attributes and references its department/faculty.
Enrollment	Stores the student–course registration relationship.
Exam	Stores each course exam details.
Result	Stores marks/grade information for an enrollment.

8. SQL IMPLEMENTATION

8.1 DDL – Data Definition Language

The following DDL is a documentation-ready implementation aligned with the supplied relational schema. Data types may be adjusted to the specific DBMS used by the student team.

```
CREATE TABLE DEPARTMENT (  
    DEPT_ID    NUMBER(5) PRIMARY KEY,  
    DEPT_NAME  VARCHAR2(50) NOT NULL,  
    DEPT_CODE  VARCHAR2(10) UNIQUE NOT NULL,  
    HOD_NAME   VARCHAR2(50) NOT NULL  
);  
  
CREATE TABLE STUDENT (  
    STUDENT_ID NUMBER(5) PRIMARY KEY,  
    ROLL_NO    VARCHAR2(20) UNIQUE NOT NULL,  
    STUDENT_NAME VARCHAR2(50) NOT NULL,  
    GENDER     VARCHAR2(10) CHECK (GENDER IN ('Male', 'Female', 'Other')),  
    DOB        DATE NOT NULL,  
    AGE        NUMBER(3),  
    EMAIL      VARCHAR2(100) UNIQUE,  
    PHONE_NO   VARCHAR2(15) UNIQUE,  
    ADDRESS    VARCHAR2(100),  
    ADMISSION_DATE DATE NOT NULL,  
    DEPT_ID    NUMBER(5) NOT NULL,  
  
    CONSTRAINT FK_STUDENT_DEPT  
        FOREIGN KEY (DEPT_ID)  
        REFERENCES DEPARTMENT(DEPT_ID),  
  
    CONSTRAINT CHK_STUDENT_AGE  
        CHECK (AGE >= 16 AND AGE <= 60)  
);
```

```
CREATE TABLE FACULTY (  
    FACULTY_ID  NUMBER(5) PRIMARY KEY,  
    FACULTY_NAME VARCHAR2(50) NOT NULL,  
    QUALIFICATION VARCHAR2(50),  
    DESIGNATION VARCHAR2(50),
```

```
EMAIL      VARCHAR2(100) UNIQUE,  
PHONE_NO   VARCHAR2(15) UNIQUE,  
JOIN_DATE  DATE,  
EXPERIENCE NUMBER(3,1),  
DEPT_ID    NUMBER(5) NOT NULL,
```

```
CONSTRAINT FK_FACULTY_DEPT  
    FOREIGN KEY (DEPT_ID)  
    REFERENCES DEPARTMENT(DEPT_ID),
```

```
CONSTRAINT CHK_EXPERIENCE  
    CHECK (EXPERIENCE >= 0)
```

```
);
```

```
CREATE TABLE COURSE (  
    COURSE_ID   NUMBER(5) PRIMARY KEY,  
    COURSE_CODE VARCHAR2(15) UNIQUE NOT NULL,  
    COURSE_NAME VARCHAR2(100) NOT NULL,  
    CREDITS     NUMBER(2) CHECK (CREDITS BETWEEN 1 AND 10),  
    SEMESTER    NUMBER(2) CHECK (SEMESTER BETWEEN 1 AND 8),  
    DEPT_ID     NUMBER(5) NOT NULL,  
    FACULTY_ID  NUMBER(5) NOT NULL,
```

```
CONSTRAINT FK_COURSE_DEPT  
    FOREIGN KEY (DEPT_ID)  
    REFERENCES DEPARTMENT(DEPT_ID),
```

```
CONSTRAINT FK_COURSE_FACULTY  
    FOREIGN KEY (FACULTY_ID)  
    REFERENCES FACULTY(FACULTY_ID)
```

```
);
```

```
CREATE TABLE ENROLLMENT (  
    ENROLLMENT_ID NUMBER(5) PRIMARY KEY,  
    STUDENT_ID    NUMBER(5) NOT NULL,  
    COURSE_ID     NUMBER(5) NOT NULL,
```

```
ENROLLMENT_DATE DATE NOT NULL,  
ACADEMIC_YEAR VARCHAR2(20) NOT NULL,  
SEMESTER NUMBER(2) CHECK (SEMESTER BETWEEN 1 AND 8),
```

```
CONSTRAINT FK_ENROLLMENT_STUDENT  
    FOREIGN KEY (STUDENT_ID)  
    REFERENCES STUDENT(STUDENT_ID),
```

```
CONSTRAINT FK_ENROLLMENT_COURSE  
    FOREIGN KEY (COURSE_ID)  
    REFERENCES COURSE(COURSE_ID)
```

```
);
```

```
CREATE TABLE EXAMS (  
    EXAM_ID    NUMBER(5) PRIMARY KEY,  
    EXAM_NAME  VARCHAR2(50) NOT NULL,  
    EXAM_DATE  DATE NOT NULL,  
    MAX_MARKS  NUMBER(3) CHECK (MAX_MARKS > 0),  
    PASSING_MARKS NUMBER(3) CHECK (PASSING_MARKS > 0)  
);
```

```
CREATE TABLE RESULT (  
    RESULT_ID    NUMBER(5) PRIMARY KEY,  
    ENROLLMENT_ID NUMBER(5) UNIQUE NOT NULL,  
    INTERNAL_MARKS NUMBER(3) CHECK (INTERNAL_MARKS BETWEEN 0 AND 100),  
    EXTERNAL_MARKS NUMBER(3) CHECK (EXTERNAL_MARKS BETWEEN 0 AND 100),  
    TOTAL_MARKS  NUMBER(3),  
    GRADE        VARCHAR2(2),  
    RESULT_STATUS VARCHAR2(10),
```

```
CONSTRAINT FK_RESULT_ENROLLMENT  
    FOREIGN KEY (ENROLLMENT_ID)  
    REFERENCES ENROLLMENT(ENROLLMENT_ID),
```

```
CONSTRAINT CHK_RESULT_STATUS  
    CHECK (RESULT_STATUS IN ('PASS', 'FAIL'));
```

```
CONSTRAINT CHK_TOTAL_MARKS
CHECK (TOTAL_MARKS BETWEEN 0 AND 200)
);
```

8.2 DML – Data Manipulation Language

```
INSERT INTO DEPARTMENT VALUES (1, 'Computer Science', 'CSE', 'Dr. Ravi Kumar');

INSERT INTO STUDENT VALUES(101, '25B11CS001', 'Aarav Sharma', 'Male', DATE '2006-05-12',
'aarav@gmail.com', '9876500001', 'Hyderabad', DATE '2025-08-01', 1);

INSERT INTO FACULTY VALUES(201, 'Dr. Ravi Kumar', 'Ph.D', 'Professor', 'ravi@univ.edu', '9888000001', DATE '2015-06-10', 11,
1);

INSERT INTO COURSE VALUES(301, 'CS101', 'Programming in C', 4, 1, 1, 201);

INSERT INTO ENROLLMENT VALUES(401, 101, 301, DATE '2025-08-05', '2025-26', 1);

INSERT INTO EXAMS VALUES(501, 'Mid Term 1', DATE '2025-09-15', 100, 40);

INSERT INTO RESULT VALUES(601, 401, 501, 38, 52, 90, 'A+', 'PASS');
```

8.3 DQL – Data Query Language

```
SELECT * FROM Student;

SELECT Student_ID, Student_Name, Email
FROM Student
WHERE Department_ID = 1;

SELECT Course_ID, Course_Code, Course_Name, Credits
FROM Course
ORDER BY Course_Name;

SELECT s.Student_Name, d.Department_Name
FROM Student s
JOIN Department d ON s.Department_ID = d.Department_ID;

SELECT s.Student_Name, c.Course_Name, e.Academic_Year, e.Semester
FROM Student s
JOIN Enrollment e ON s.Student_ID = e.Student_ID
JOIN Course c ON e.Course_ID = c.Course_ID;
```

9. SQL QUERIES & SCREENSHOTS

The following query set is prepared to satisfy the documentation requirement for a 30–40 query SQL section. The examples cover selection, filtering, joins, aggregation, subqueries, updates, deletion, and reporting. Exact outputs depend on the records inserted in the final project database.

Q1. Display student names and roll numbers.

```
SELECT ROLL_NO, STUDENT_NAME FROM STUDENT ;
```

Output:

ROLL_NO	STUDENT_NAME
25B11CS001	Aarav Sharma
25B11IT002	Ananya Reddy
25B11EC003	Rahul Kumar
25B11EE004	Sneha Rao
25B11ME005	Vikram Reddy
25B11CV006	Priya Singh
25B11AI007	Karthik Rao
25B11DS008	Keerthi Reddy
25B11CS009	Arjun Varma
25B11BT010	Divya Sharma
25B11CH011	Rohit Kumar
25B11AE012	Megha Rao
25B11AU013	Sanjay Reddy
25B11RO014	Pooja Devi
25B11IS015	Manoj Kumar

Q2. Display students from the Computer Science department.

```
SELECT STUDENT_NAME, ROLL_NO, DEPT_ID FROM STUDENT  
WHERE DEPT_ID = 1;
```

Output:

STUDENT_NAME	ROLL_NO	DEPT_ID
Aarav Sharma	25B11CS001	1

Q3. Display different genders available in the student table.

SELECT DISTINCT GENDER FROM STUDENT;

Output:

GENDER
Male
Female

Q4. Find the total number of students.

SELECT COUNT(*) AS TOTAL_STUDENTS FROM STUDENT;

Output:

TOTAL_STUDENTS
15

Q5. Display students in alphabetical order.

SELECT STUDENT_NAME, ROLL_NO FROM STUDENT
ORDER BY STUDENT_NAME;

Output:

STUDENT_NAME	ROLL_NO
Aarav Sharma	25B11CS001
Ananya Reddy	25B11IT002
Arjun Varma	25B11CS009
Divya Sharma	25B11BT010
Karthik Rao	25B11AI007
Keerthi Reddy	25B11DS008
Manoj Kumar	25B11IS015
Megha Rao	25B11AE012
Pooja Devi	25B11RO014
Priya Singh	25B11CV006
Rahul Kumar	25B11EC003
Rohit Kumar	25B11CH011
Sanjay Reddy	25B11AU013
Sneha Rao	25B11EE004
Vikram Reddy	25B11ME005

Q6. Display courses with 4 credits.

```
SELECT COURSE_NAME, CREDITS FROM COURSE  
WHERE CREDITS = 4;
```

Output:

COURSE_NAME	CREDITS
Programming in C	4
Database Management Systems	4
Electrical Machines	4
Thermodynamics	4
Structural Engineering	4
Artificial Intelligence	4
Data Analytics	4
Chemical Process Engineering	4
Aircraft Design	4
Automobile Engineering	4
Robotics Fundamentals	4

Q7. Display female students with name and roll no.

```
SELECT STUDENT_NAME, ROLL_NO FROM STUDENT  
WHERE GENDER = 'Female';
```

Output:

STUDENT_NAME	ROLL_NO
Ananya Reddy	25B11IT002
Sneha Rao	25B11EE004
Priya Singh	25B11CV006
Keerthi Reddy	25B11DS008
Divya Sharma	25B11BT010
Megha Rao	25B11AE012
Pooja Devi	25B11RO014

Q8. Students who scored above the average marks

```
SELECT RESULT_ID, TOTAL_MARKS  
FROM RESULT  
WHERE TOTAL_MARKS > (SELECT AVG(TOTAL_MARKS)  
FROM RESULT);
```

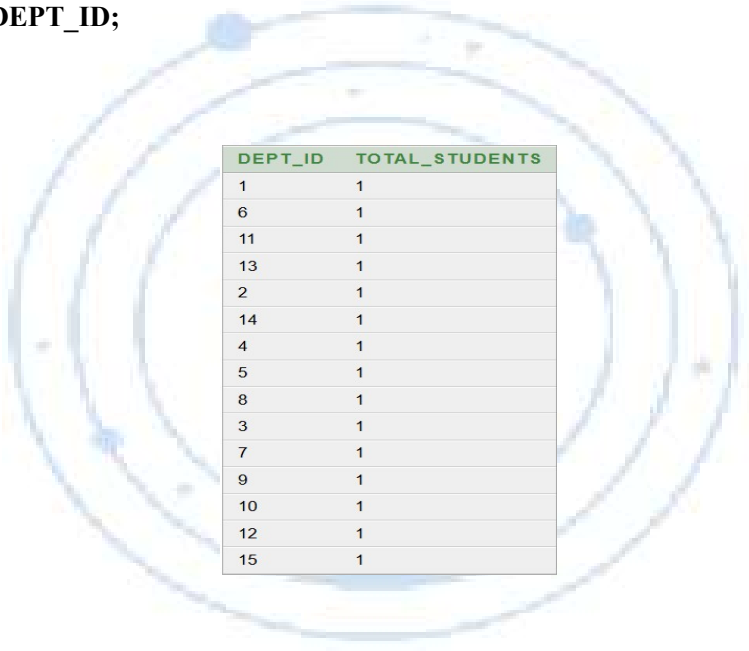
Output:

RESULT_ID	TOTAL_MARKS
601	90
602	83
603	77
605	86
607	95
608	81
610	86
612	75
614	93

Q9. The number of students in each department.

```
SELECT DEPT_ID, COUNT(*) AS TOTAL_STUDENTS  
FROM STUDENT  
GROUP BY DEPT_ID;
```

Output:



DEPT_ID	TOTAL_STUDENTS
1	1
6	1
11	1
13	1
2	1
14	1
4	1
5	1
8	1
3	1
7	1
9	1
10	1
12	1
15	1

Q10. Highest Marks

```
SELECT MAX(TOTAL_MARKS) AS HIGHEST_MARKS  
FROM RESULT;
```

Output:

HIGHEST_MARKS
95

Q11. The average marks.

```
SELECT AVG(TOTAL_MARKS) AS AVERAGE_MARKS  
FROM RESULT;
```

Output:

AVERAGE_MARKS
74.2

Q12.Join Student - Department.

```
SELECT S.STUDENT_NAME, D.DEPT_NAME
FROM STUDENT S
JOIN DEPARTMENT D
ON S.DEPT_ID = D.DEPT_ID;
```

Output:

STUDENT_NAME	DEPT_NAME
Aarav Sharma	Computer Science
Ananya Reddy	Information Technology
Rahul Kumar	Electronics and Communication
Sneha Rao	Electrical Engineering
Vikram Reddy	Mechanical Engineering
Priya Singh	Civil Engineering
Karthik Rao	Artificial Intelligence
Keerthi Reddy	Data Science
Arjun Varma	Cyber Security
Divya Sharma	Biotechnology
Rohit Kumar	Chemical Engineering
Megha Rao	Aerospace Engineering
Sanjay Reddy	Automobile Engineering
Pooja Devi	Robotics Engineering
Manoj Kumar	Information Science

Q13. Join - Course and Faculty.

```
SELECT C.COURSE_NAME, F.FACULTY_NAME
FROM COURSE C
JOIN FACULTY F
ON C.FACULTY_ID = F.FACULTY_ID;
```

Output:

COURSE_NAME	FACULTY_NAME
Programming in C	Dr. Ravi Kumar
Database Management Systems	Dr. Suresh Rao
Digital Electronics	Dr. Priya Sharma
Electrical Machines	Dr. Anand Kumar
Thermodynamics	Dr. Ramesh Babu
Structural Engineering	Dr. Lakshmi Devi
Artificial Intelligence	Dr. Kiran Reddy
Data Analytics	Dr. Meena Rao
Cyber Security	Dr. Arjun Singh
Biotechnology Basics	Dr. Kavya Rao
Chemical Process Engineering	Dr. Naveen Kumar
Aircraft Design	Dr. Vikram Singh
Automobile Engineering	Dr. Mohan Rao
Robotics Fundamentals	Dr. Swathi Reddy
Information Systems	Dr. Pavan Kumar

Q14.Join - Student and course

Arjun Varma	Cyber Security
Divya Sharma	Biotechnology Basics
Rohit Kumar	Chemical Process Engineering
Megha Rao	Aircraft Design
Sanjay Reddy	Automobile Engineering
Pooja Devi	Robotics Fundamentals
Manoj Kumar	Information Systems

Q15.Having

Output:

GRADE	AVERAGE_MARKS
A+	92.666666666666666666666666666667
A	84

11. CONCLUSION

The University Academic Management Database Application provides a structured relational database solution for managing core academic information. The design organizes departments, students, faculty, courses, enrollments, and results into related relations and applies primary keys, foreign keys, uniqueness rules, NOT NULL conditions, and CHECK constraints to support data integrity. The normalization process up to 3NF reduces redundancy and update anomalies. The SQL implementation and query set demonstrate how the database can support insertion, modification, retrieval, aggregation, and academic reporting. The project therefore provides a suitable DBMS cornerstone implementation that can be extended into a complete academic information system.

12. FUTURE ENHANCEMENTS

- Role-based access for administrators, faculty, and other authorized users.
- Attendance and leave management.
- Fee and scholarship management.
- Timetable and classroom allocation.
- Automated report and analytics dashboards.
- Backup, recovery, audit logging, and stronger database security.



REFERENCES

1. Oracle Database Documentation – Oracle SQL and relational database concepts.
2. Database Management System Course Materials - Concepts related to relational databases, normalization, SQL, keys, and constraints.
3. Oracle SQL Developer - Used for database creation, SQL execution, and query testing.
4. Aditya University DBMS Project Guidelines and Course Material – Used as the academic basis for the project structure and database concepts.
5. Project Database Design and SQL Implementation - Gym Membership Management System developed as part of the DBMS Cornerstone Project

Reference Books:

1. Database Systems, Ramez Elmasri, Shamkant B. Navathe, 7th Edition, Pearson, ISBN: 9780137504277.
2. Database Systems, Carlos Coronel, Steven Morris, Peter Rob, 10th Edition, Cengage, ISBN: 978-0357517864.
3. Introduction to Database Systems, C J Date, Pearson, 9th Edition, ISBN: 978-0133970777.

Textbooks:

1. Database System Concepts, Abraham Silberschatz, Henry Korth, and S. Sudarshan, McGrawHill, 8th Edition, ISBN-13: 978-1260230508.
2. Database Management Systems, Raghurama Krishnan, Johannes Gehrke, Mc Graw-Hill, 4th Edition, ISBN: 978-1260091195.